



Know More About

Do you know how lots of data is crammed into a much smaller zipped file?

Samir Makwana

Compression is widely used to save storage space and for faster data transmission over networks and the Internet. A large file (or a number of files) is shrunk into a compact file that can be “expanded” later to get back the original files. In the case of lossy compression (we’ll come to that in a minute), you get a much smaller MP3 file from a huge WAV file (for example). So the question is: how does data compression work?

Lossless Compression

Let’s talk about lossless compression first, basically what happens when a number of files are zipped. Here, you get back all the original data when you unzip it. The simple fact is that data files have redundancy—the same information is represented repetitively. For example, in a text file, pronouns, prepositions, punctuation, and such are repeated throughout the document. The redundancy can be removed through compression by listing the bits of repeated information or common elements (for example, patterns or shapes in the case of image files) *once*, instead of listing them again and again.

One of the simplest ways to understand the working of compression is by considering a text file. In the phrase “A penny saved is a penny earned,” each letter, space and punctuation mark occupies one unit of space in the storage media. This file would occupy 32 units of space—comprising 25 letters, 6 spaces, and 1 full stop.

If we look for redundancies, the words “a” and “penny”, which are repeated twice each, can be replaced with the numbers 1 and

2 using a coding scheme. So this scheme is: A=1, Penny=2.

The coding scheme thus consists of 8 characters (6 letters and 2 numbers). This scheme itself has to be saved in the resulting compressed file so the compression program knows how to unzip the data.

The phrase, after applying the coding scheme, would read “1 2 saved is 1 2 earned,” which would occupy 23 units of space. This is saved by the compression program along with the coding scheme to



An aggressively-compressed JPEG—notice the ugly blocks

form a compressed file. The size of this file would be 31 units, which consist of 23 characters (with spaces) of the new coded phrase and 8 characters (numbers and letters) of the coding scheme.

So, as compared to the original file size, that is, 32 units, the compressed file size now requires 31 units of space. Here, the dictionary takes up comparatively more space since the original phrase is small. But in the case of a much larger text file, the overhead will reduce: the dictionary will be comparatively smaller, and there will be more repeated patterns in the data. For instance, in the above example, if the next sentence were “So don’t waste a penny!”, we add only 3 items to the dictionary instead of 5.

Lossless Compression uses a number of algorithms such as run length coding, the Burrows-Wheeler transform, dictionary coders, prediction by partial matching, context mixing, and entropy coding. Of these, Lempel and Ziv’s (LZ) dictionary-based algorithm for data compression is the most popular and widely-used.

Lossy Compression

When you’re talking to someone on the phone, what matters more—the clarity with which you can hear their words, or whether the phone line reproduces the deep baritone of his voice? It’s this same premise that gave people the idea of *lossy compression*—compressing files in a way that eliminates some of the “frills” from the file, but keeping its heart intact. A good example is JPEG compression—try saving a JPEG at maximum compression through a program like IrfanView—it looks awful, but you can still tell that it’s a dog or a pig or whatever.

The advantage of lossy compression over lossless is the fact that with the freedom of destroying some information from the file itself, you can achieve smaller file sizes. Such stunts aren’t tried with things like text compression, of course—all the data is essential! Lossy compression finds itself used most with images, audio and video—all one needs to do is provide a threshold beyond which all information will be destroyed. Once compressed, you can’t regain the quality of the original file, and if you compress an already compressed file, you lose even more quality, no matter how generous you are with the threshold.

We wish we could talk here about how an MP3 file is created from a WAV file, but space just doesn’t permit it! ■

samir_makwana@thinkdigit.com